

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב

67130 – מבוא למדעי המחשב בדגש על תכנות פרוצדורלי

תאריך: 26.5.2006

משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' - תשס"ו – 2005-2006

המורה: פרופ' ג'ף רוזנשיין

הנחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (33 נק').
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתבו באופן מודולארי, אל תכפילו קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה. עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בקלט (למשל מותר להקצות מערך בן תא אחד פעם אחת).
- בכל השאלות מותר להניח שהקלט חוקי (ומקיים את תנאי השאלה).

שאלה 1

- מפאת נדיבותו הרבה, החליט קבלן ידוע ובעל שם לחלק מספר דירות לכמה ממקורביו. מקורבים פחותי ערך יקבלו דירה ובה חדר שירותים אחד (בחזקת "שלח לחמך על פני המים"). מקורבים הנושאים במשרות מפתח בעריות המקומיות יקבלו דירה עם שני חדרי שירותים (אחד בחדר ההורים). סגני ראש עיר יקבלו דירת יוקרה ובה שלושה חדרי שירותים (כיאה למעמדם), וראשי ערים יזכו בדירת פאר ובה לא פחות מארבעה חדרי שירותים (שכן רק הגיוני ומתחייב הדבר).
- בכדי ליעיל את תהליך החלוקה, החליט הקבלן לייצג דירה באמצעות המחלקה Flat שבה מוגדרת הפונקציה `getToiletNumber()` המחזירה את מספר חדרי השירותים שבדירה (בין 1 ל-4). הדירות קובצו למערך אחד אשר יש למיין לפי מספר חדרי השירותים שבדירות, בסדר עולה.
- כיוון שזמנו של הקבלן יקר עד מאוד, על פונקציה המיין לרוץ בסדר גודל זמן ריצה $O(n)$ (המקרה הגרוע).
- בנוסף, מטעמים ברורים, יש לשמור על הנושא בחשאיות רבה ככל שניתן, ולכן בזמן המיין מותר לקרוא ל-`getToiletNumber()` לכל היותר פעם אחת בעבור כל אובייקט במערך.

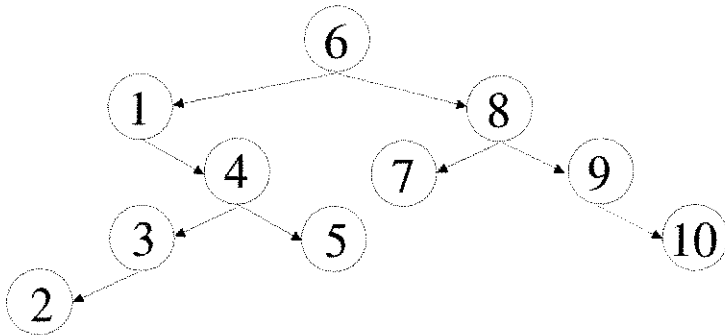
א. (28 נק'). כתוב פונקציה סטטית (השייכת למחלקה ללא data-members):
`public static void sortFlats(Flat[] flats);`
המקבלת את המערך הנ"ל וממיינת אותו כנדרש. שים לב לסעיף 11 בתחילת הבחינה.

ב. (5 נק'). הסבר מדוע סיבוכיות פתרוןך בסעיף א' היא אכן $O(n)$ ושכנע ש-`getToiletNumber()` אכן נקראת לכל היותר פעם אחת בעבור כל אובייקט.

שאלה 2

- בתכנית מוגדר עץ חיפוש בינארי באמצעות המבנה הבא:

```
class Node {
    int data;
    Node left;
    Node right;
}
```



- בהינתן קודקוד n בעץ חיפוש בינארי כנ"ל, נגדיר את העוקב (successor) של n להיות הקודקוד s כך ש- s מכיל את הערך המינימלי בעץ שמקיים: $n.data < s.data$.
- לדוגמא, בעץ משמאל, $i+1$ הוא העוקב של i (כלומר 2 הוא העוקב של 1, 3 העוקב של 2, וכו').
- ל- 10 (הערך הגדול בעץ) אין עוקב ולכן מגדירים את עוקבו להיות null.
- מניחים כי כל הערכים בעץ החיפוש שונים זה מזה.

א. (18 נק'). כתוב פונקציה סטטית (השייכת למחלקה ללא data-members):

```
public static Node successor(Node root, Node n);
```

המקבלת שורש של עץ חיפוש כנ"ל (root) וכן קודקוד כלשהו בעץ זה (n) ומחזירה את העוקב של n. שים לב: מותר (אך לא מתחייב) להניח שב- Node קיים data-member נוסף מסוג Node בשם parent, אשר מצביע על קודקוד האב. (למשל ה- parent של 4 הוא 1, של 7 הוא 8, ושל 6 הוא null). הנחה זו אינה הכרחית, אך פתרונות שמשתמשים בה קבילים באותה מידה (למרות שפתרון אחד יעיל מהאחר). שימו לב לסעיף 11 בתחילת הבחינה.

ב. (10 נק'). באמצעות הפונקציה שמימשת בסעיף א', כתוב מחלקה בשם TreeIterator המממשת את הממשק:

```
interface Iterator {
    boolean hasNext(); // is there another item in the sequence?
    int getNext() throws NoSuchElementException; // return next element in the sequence
}
```

כך של- TreeIterator יש constructor המקבל שורש של עץ חיפוש כנ"ל:

```
Node root;
```

וניתן להשתמש במחלקה זו באופן הבא:

```
Iterator iter = new TreeIterator(root);
while( iter.hasNext() )
    System.out.println( iter.getNext() );
```

כך שבתום הלולאה יודפסו כל איברי העץ, לפי הסדר, מהקטן לגדול.

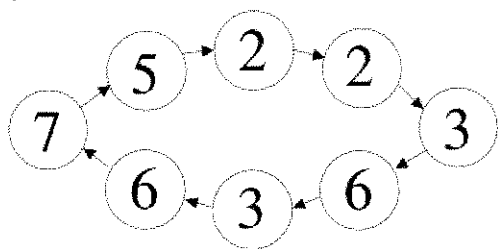
ג. (3 נק'). נניח שבעץ ששורשו root יש k איברים, מהי סיבוכיות זמן הריצה של הפונקציה successor (המקרה הגרוע)? הסבר.

ד. (2 נק'). מהי סיבוכיות זמן הריצה של הלולאה שבסעיף ב'? הסבר.

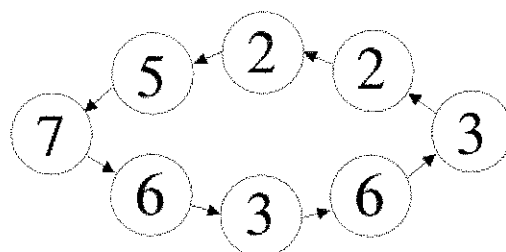
שאלה 3

- בתכנית מוגדר רשימה משורשרת מעגלית באמצעות המבנה הבא:

```
class Node {
    int data;
    Node next;
}
```



- רשימה מעגלית מתאפיינת בכך שהתא "האחרון" שבה מצביע על התא "הראשון". כיוון שכך, למעשה, אין תא "ראשון" או "אחרון", וכל תא יכול לשמש כ"ראש" הרשימה. משמאל מוצגת דוגמה לרשימה מעגלית.
- רשימה ריקה מיוצגת, כרגיל, ע"י null. אם יש ברשימה מעגלית איבר אחד בלבד אזי הוא מצביע על עצמו (head = head.next). ואם יש שני איברים אזי הם מצביעים זה על זה.
- הנחיות לשאלה זו:
 - אסור להשתמש ברקורסיה.
 - אסור להפוך את הרשימה המעגלית לרשימה רגילה (ולפתור את השאלה בעבור הרשימה הרגילה ואז להפוך חזרה את הרשימה למעגלית).
 - שימו לב לסעיפים 10 ו- 11 בתחילת הבחינה.



א. (14 נק'). כתוב פונקציה סטטית (השייכת למחלקה ללא data-members):

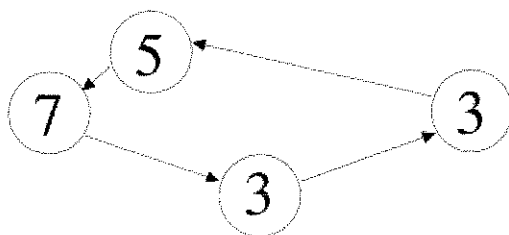
```
public static void reverse(Node head);
```

המקבלת רשימה כנ"ל והופכת את סדר ההצבעות שבה. לדוגמה, התוצאה של הפעלת reverse על הרשימה המעגלית הנ"ל מוצגת משמאל.

ב. (15 נק'). כתוב פונקציה סטטית (השייכת למחלקה ללא data-members):

```
public static Node removeEven(Node head);
```

המקבלת רשימה מעגלית כנ"ל ומוחקת ממנה את כל האיברים הזוגיים. הפונקציה מחזירה את "ראש" הרשימה המעגלית המעודכנת (שעשוי להיות שונה מה- head המקורי אם ה- data שלו זוגי). אין חשיבות לזהות האיבר המוחזר כל עוד הוא שייך לרשימה המעודכנת (כלומר כל עוד ערכו אי זוגי). מובן שאם כל איברי הרשימה המקורית הינם זוגיים, הפונקציה תחזיר את הרשימה הריקה. דוגמה: התוצאה של הפעלת removeEven על הרשימה המוצגת בסעיף א' היא:

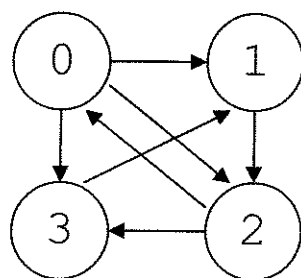


ג. (2 נק'). מהי סיבוכיות זמן הריצה של הפונקציה reverse (המקרה הגרוע)? הסבר.

ד. (2 נק'). מהי סיבוכיות זמן הריצה של הפונקציה removeEven (המקרה הגרוע)? הסבר.

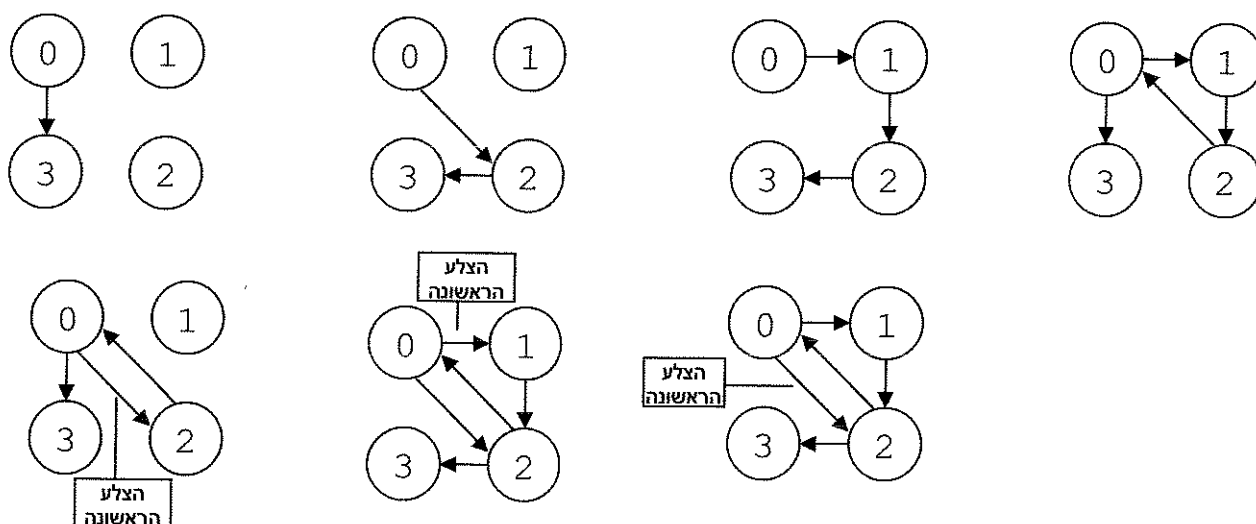
שאלה 4

- נתונה קבוצת קודקודים: $V = \{0, 1, 2, \dots, n-1\}$.
- נגדיר גרף על קבוצת הקודקודים V באמצעות מטריצת שכנויות: מערך דו-מימדי $(n \times n)$ של בוליאנים בשם $graph$, כך ש- $graph[i][k]$ הוא $true$ אם ורק אם יש צלע מהקודקוד i לקודקוד k .
- לדוגמא, מטריצת השכנויות $(n=4)$ של הגרף משמאל נתונה מימין:



המערך graph	שורות (i)	עמודות (k)			
		0	1	2	3
	0	false	true	true	true
	1	false	false	true	false
	2	true	false	false	true
	3	false	true	false	false

- א. (28 נק'). כתוב פונקציה סטטית (השייכת למחלקה ללא data-members):
- ```
public static int count(boolean graph[], int start, int end);
```
- אשר מקבלת מטריצת שכנויות  $(graph)$ , קודקוד התחלה  $(start)$ , וקודקוד סיום  $(end)$ , ומחזירה את מספר המסלולים בין קודקוד ההתחלה לבין קודקוד הסיום כך ששום צלע לא מופיעה במסלול נתון יותר מפעם אחת. לדוגמא, ערך ההחזרה של  $count$  על הגרף הנ"ל עם  $start=0$  ו-  $end=3$  הוא 7:



הבהרות:

- כרגיל, ניתן להניח כי הקלט תקין:  $graph$  מייצג מטריצת שכנויות חוקית ו-  $start$  ו-  $end$  הם שני קודקודים בגרף.
- מסלול מוגדר להיות רצף של קודקודים, כך שבין כל שני קודקודים עוקבים יש צלע. קודקוד הסיום יכול להופיע רק פעם אחת בכל מסלול (בקצהו), אולם כל קודקוד אחר יכול להופיע מספר רב של פעמים באותו מסלול (ובלבד שכל צלע תופיע פעם אחת לכל היותר).
- לשם נוחות נגדיר שהפונקציה  $count$  תחזיר 1 בעבור המקרה שבו  $start = end$ .
- שימו לב לסעיף 11 שבתחילת הבחינה.

- ב. (5 נק'). מהי סיבוכיות זמן הריצה של הפונקציה  $count$  (המקרה הגרוע)? הסבר.

**בהצלחה!**